

Interim Report: Medical Telegram Data Warehouse

Interim Report: Medical Telegram Data Warehouse

Project Overview

Business Objectives

Business Questions the Architecture Enables

- Which product categories are most frequently promoted? → Analyzed through message text classification and channel_type dimension
- What are the peak engagement times for medical product promotions? → Enabled through dim_date attributes (day_of_week, is_weekend, hour) joined with fct_messages view_count
- Which channels have the highest audience engagement? → Calculated via dim_channels.avg_views and total_posts metrics
- How does image content affect message engagement? → Measured through has_image_flag in fct_messages correlated with view_count and forward_count
- What are the temporal trends in product availability? → Tracked through date dimension analysis of message posting frequency
- Which product descriptions drive the most forwards? → Analyzed through message_length and message_text patterns in fct_messages

Data Lake Structure

Directory Organization

Data Lake Design Decisions

- Messages are partitioned by date (YYYY-MM-DD) to optimize query performance for time-based analysis
- Each date partition contains JSON files for each channel scraped on that date
- This structure allows efficient data loading and querying for specific time periods

- Images are stored in a channel-specific directory structure
- Each image is named by its message ID for easy reference back to the message data
- This separation of text and binary data improves storage efficiency and access patterns
- Raw data is stored in JSON format to preserve the original structure from the Telegram API
- JSON is human-readable and easily parsable for downstream processing
- Maintains flexibility for schema evolution as API responses may change

Data Fields Collected

Field	Description	Type
-----	-----	-----
message_id	Unique identifier for each message	Integer
channel_name	Name of the Telegram channel	String
message_date	Timestamp of the message	ISO 8601 datetime
message_text	Full text content (product names, prices, descriptions)	String
has_media	Whether the message contains media	Boolean
image_path	Path to downloaded image (if applicable)	String
views	Number of views on the message	Integer
forwards	Number of times the message was forwarded	Integer

Star Schema Design

Overview

Star Schema Diagram

Business Value of Star Schema Design

- Fast Channel Performance Queries: dim_channels pre-aggregates metrics (avg_views, total_posts) for instant channel comparison without complex joins
- Temporal Analysis Efficiency: dim_dates provides pre-calculated date attributes (weekend flags, quarters) for quick trend analysis without date manipulation
- Scalable Message Analysis: fct_messages stores individual message metrics that can be aggregated by any dimension (channel, date, or future dimensions like product category)
- Flexible Schema Evolution: New dimensions (e.g., dim_products, dim_locations) can be added without disrupting existing fact table structure

- Optimized for BI Tools: Star schema structure is compatible with most BI and visualization tools for business user self-service analytics

Dimension Tables

dim_channels

- channel_key: Surrogate key for unique channel identification
- channel_name: Original Telegram channel name
- channel_type: Classification (Pharmaceutical/Cosmetics/Medical)
- first_post_date: Date of first message from this channel
- last_post_date: Date of most recent message from this channel
- total_posts: Count of all messages from this channel
- avg_views: Average view count across all messages

dim_dates

- date_key: Surrogate key (YYYYMMDD format)
- full_date: Complete date value
- day_of_week: Numeric day of week (1-7)
- day_name: Day name (Monday, Tuesday, etc.)
- week_of_year: Week number within the year
- month: Numeric month (1-12)
- month_name: Month name
- quarter: Quarter of year (Q1-Q4)
- year: Year value
- is_weekend: Boolean flag for weekend days

Fact Table

fct_messages

- message_id: Unique message identifier (natural key)
- channel_key: Foreign key to dim_channels
- date_key: Foreign key to dim_dates
- message_text: Full message content
- message_length: Character count of message text

- view_count: Number of views
- forward_count: Number of forwards
- has_image_flag: Boolean indicating image presence

Star Schema Benefits

Data Quality Issues and Solutions

Issue 1: API Rate Limiting

- Implemented error handling in the scraper to catch rate limit exceptions
- Added logging to track when rate limits are encountered
- Structured the scraper to be resumable - can be run multiple times without duplicating data
- Used async/await pattern to handle network delays gracefully

Issue 2: Private/Inaccessible Channels

- Added specific error handling for ChannelPrivateError and ChannelInvalidError
- Implemented graceful degradation - if one channel fails, others continue to be scraped
- Logged which channels are inaccessible for later review
- Configurable channel list in .env allows easy removal of problematic channels

Issue 3: Incomplete Message Data

- Implemented default values for missing numeric fields (0 for views/forwards)
- Empty strings for missing text fields
- Boolean flags to indicate presence/absence of media
- All data quality issues are logged for review
- Staging layer in dbt will apply additional validation and cleaning

Issue 4: Image Download Failures

- Individual try-catch blocks for each image download
- Failed image downloads don't stop the overall scraping process
- Image path set to null if download fails
- Detailed error logging for troubleshooting

Issue 5: Date Consistency

- Telegram API returns ISO 8601 format which is standardized
- All dates stored in UTC for consistency
- Date dimension table will provide localized date attributes in the transformation layer
- Staging models will validate date ranges and identify anomalies

Issue 6: Duplicate Messages

- Data lake partitioned by date prevents accidental overwrites
- Each JSON file contains messages for a specific date and channel
- Future implementation will include deduplication logic in the staging layer
- Natural keys (message_id + channel_name + date) will be used to identify duplicates

Current Progress

Completed (Task 1)

- ■ Project structure and environment setup
- ■ Telegram API integration with Telethon
- ■ Data scraper implementation
- ■ Image download functionality
- ■ Data lake storage with partitioned structure
- ■ Logging and error handling
- ■ Git repository initialization and code push

In Progress (Task 2)

- ■ PostgreSQL database setup
- ■ Raw data loading script
- ■ dbt project initialization
- ■ Staging models development
- ■ Star schema implementation
- ■ Data quality tests

Pending (Tasks 3-5)

- ■ YOLO object detection for image enrichment

- ■ FastAPI analytical API development
- ■ Dagster pipeline orchestration

Technical Stack

- Scraping: Python, Telethon
- Storage: JSON files (data lake), PostgreSQL (data warehouse)
- Transformation: dbt (Data Build Tool)
- Enrichment: YOLOv8 (object detection)
- API: FastAPI
- Orchestration: Dagster
- Version Control: Git, GitHub

Detailed Next Steps and Anticipated Challenges

Task 2: Data Modeling and Transformation with dbt

- Install and configure PostgreSQL database
- Create raw schema and telegram_messages table
- Develop Python script to load JSON files from data lake to PostgreSQL
- Anticipated Challenges:
- Install dbt-postgres and initialize dbt project
- Configure profiles.yml for PostgreSQL connection
- Set up project structure (models/staging, models/marts, tests/)
- Anticipated Challenges:
- Create stg_telegram_messages.sql model
- Implement data type casting (dates, integers)
- Add calculated fields (message_length, has_image_flag)
- Filter invalid records (empty messages, nulls)
- Anticipated Challenges:
- Create dim_channels dimension table
- Create dim_dates dimension table with comprehensive date attributes
- Create fct_messages fact table with foreign keys
- Implement surrogate key generation
- Anticipated Challenges:

- Add unique and not_null tests on primary keys
- Implement relationships tests on foreign keys
- Create custom data tests (assert_no_future_messages, assert_positive_views)
- Anticipated Challenges:

Task 3: Data Enrichment with YOLO Object Detection

- Install ultralytics library
- Download YOLOv8 nano model (yolov8n.pt)
- Test object detection on sample images
- Anticipated Challenges:
- Create src/yolo_detect.py script
- Scan downloaded images in data/raw/images/
- Run YOLO detection on each image
- Record detected objects with confidence scores
- Save results to CSV file
- Anticipated Challenges:
- Load YOLO detection results into PostgreSQL
- Create dim_objects dimension table
- Add object detection results to fact table or create bridge table
- Update dbt models to include enriched data
- Anticipated Challenges:

Task 4: Analytical API Development with FastAPI

- Install FastAPI, uvicorn, and dependencies
- Create project structure (api/main.py, api/database.py, api/schemas.py)
- Implement database connection pooling
- Anticipated Challenges:
- Create endpoints for channel metrics
- Create endpoints for temporal analysis
- Create endpoints for message search
- Implement pagination and filtering
- Anticipated Challenges:
- Generate OpenAPI documentation

- Add example requests/responses
- Implement unit tests for endpoints
- Add logging and monitoring
- Anticipated Challenges:

Task 5: Pipeline Orchestration with Dagster

- Install Dagster and dependencies
- Initialize Dagster project
- Configure Dagster UI
- Anticipated Challenges:
- Define assets for scraping, transformation, enrichment
- Create job dependencies and schedules
- Implement error handling and retries
- Anticipated Challenges:
- Deploy Dagster to production environment
- Set up monitoring and alerting
- Implement backfill strategies
- Anticipated Challenges:

Cross-Cutting Considerations

- Maintain comprehensive code comments
- Keep README files updated for each component
- Document API endpoints and data schemas
- Create architecture diagrams
- Unit tests for transformation logic
- Integration tests for API endpoints
- End-to-end tests for complete pipeline
- Data quality tests in dbt
- Database indexing for frequently queried columns
- Caching strategies for API responses
- Batch processing for large datasets
- Query optimization for complex joins
- Secure credential management (environment variables, secrets)

- API authentication and authorization
- Data encryption at rest and in transit
- Regular security audits

Conclusion